Assignment-4

Non-Linear & Advanced Data Structures

IMPLEMENTATION OF TREES AND GRAPHS

Institute Name- K.R. Mangalam University

Course Name: Data Structures

Program Name: B. Tech Computer Science and Engineering (AI and ML)

Section: A

Faculty Name: Dr. Neetu Chauhan

IMPLEMENTING TREES AND GRAPHS USING BST, BFS, DFS AND HASHING

By: Mehul Srivastava

Roll No.- 2501730149

1. Introduction:

In this assignment, I implemented a Data Management Mini Toolkit (DMMT) using Python.
The objective was to understand and implement important non-linear and advanced data structures used in real-world systems.

The project consists of three main components:

- Binary Search Tree (BST)
- Graph (BFS and DFS traversal)
- Hash Table with Separate Chaining

These structures are widely used in databases, networking, and fast data retrieval systems.

2. Binary Search Tree (BST):

A Binary Search Tree is a hierarchical data structure in which:

- Left subtree contains values smaller than the root
- Right subtree contains values greater than the root

2.1. Operations Implemented:

Insertion: Adds elements at correct position

Search: Finds whether an element exists

Deletion: Handles:

- Leaf node
- Node with one child
- Node with two children

Inorder Traversal: Displays elements in sorted order

2.2. Why Inorder traversal prints a BST in sorted order?

Inorder traversal follows: Left → Root → Right. This ensures values are printed in sorted order.

2.3. Time Complexity:

- Average Case: O(log n)
- Worst Case: O(n) (when tree becomes skewed)

3. Graph Representation (Adjacency List):

A graph is a collection of nodes (vertices) connected by edges. In this assignment, the graph is represented using an adjacency list, where each node stores its connected neighbours along with weights.

3.1. Breadth First Search (BFS):

- Uses a queue (FIFO)
- Traverses' nodes level by level
- Used in: finding shortest path in unweighted graph

3.2. Depth First Search (DFS):

- Uses recursion stack (LIFO)
- Traverses' nodes depth-wise
- Used in: Used for cycle detection

4. Hash Table:

A hash table stores key-value pairs and allows fast access using a hash function.

4.1. What is Collision:

A collision occurs when multiple keys map to the same index.

4.2. Collision Handling, why separate chaining helps?

Used Separate Chaining, where:

- Each index stores a list of key-value pairs
- Multiple elements can exist in the same bucket

4.3. Time Complexity:

Average Case: O(1)

Worst Case (collisions happening): O(n)

5. Conclusion:

This assignment helped in understanding how different data structures work in real-life applications.
It also demonstrated how searching, traversal, and fast lookup are implemented efficiently.